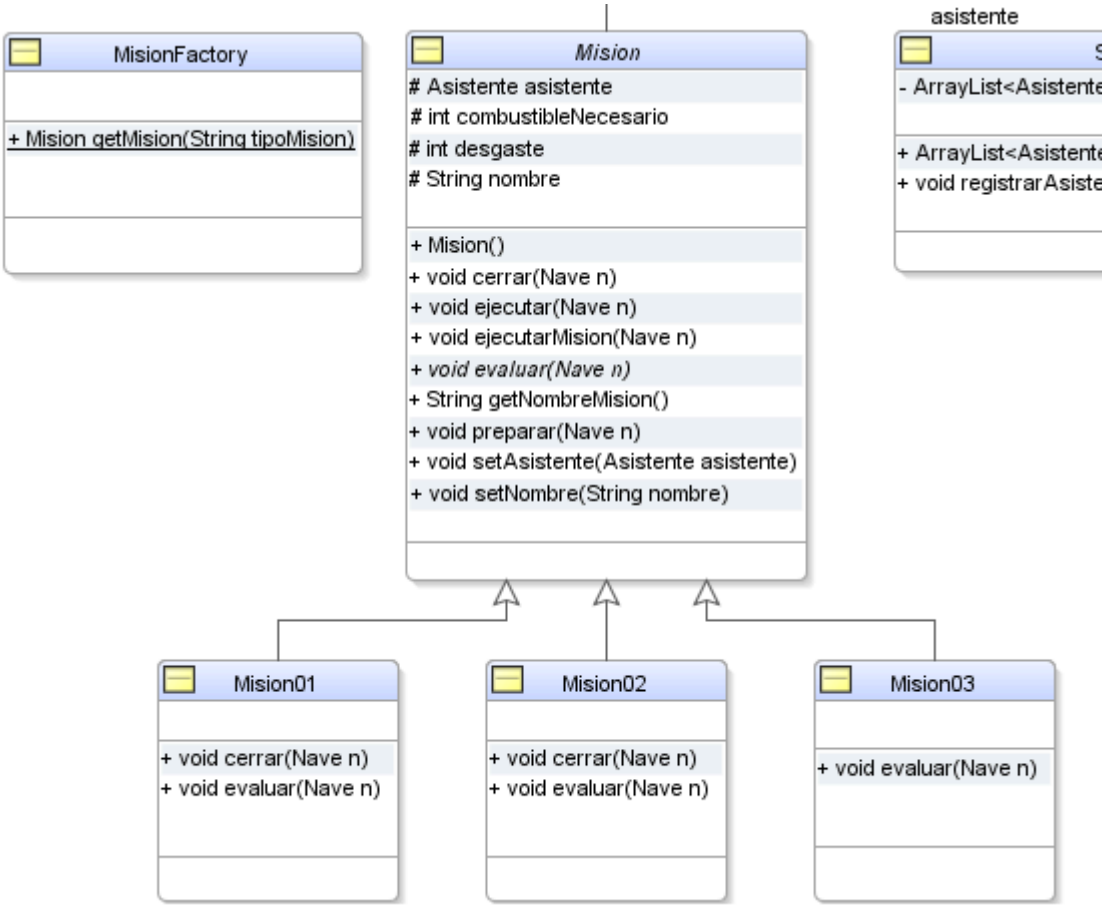
Informe y Documentación de Patrones del TP Grupal de Programación C

A continuación se describirán el uso de cada patrón visto en clase y utilizado en el proyecto:

Template

Para poder diseñar las distintas misiones de forma que todas siguieran la misma serie de pasos utilizamos el patrón de diseño Template que nos permite modelar una secuencia de métodos a ejecutar en cada misión, permitiendo que cada una modifique únicamente los pasos que hagan falta y dejando los pasos en común iguales para todas las misiones.



Los métodos cerrar y ejecutar son iguales para los tres tipos de misión. El método evaluar es abstracto permitiendo que cada misión lo ejecute con lo que necesite. Por último el método cerrar es un método vacío que se sobreescribe únicamente en las misiones 1 y 2.

Factory

Utilizamos el patrón Factory en dos partes del proyecto, en la creación de Naves y la creación de Misiones.

```
public class NaveFactory {
    /**
     * Metodo que devuelve una nave dependiendo del tipo de nave que se le pase por parametro.
     * <b>PRE:</b>
     * - tipoNave != null
     *
     * @param tipoNave Tipo de nave (Carguero", "Combate", "Exploradora").
     * @return Una instancia de Nave correspondiente al tipo especificado, o null si el tipo no es válido.
     */
    public static Nave getNave(String tipoNave) {
        switch (tipoNave) {
            case "Carguero":
                return new Carguero();
            case "Combate":
                return new Combate();
            case "Exploradora":
                return new Exploradora();
            default:
                return null;
        }
    }
}
```

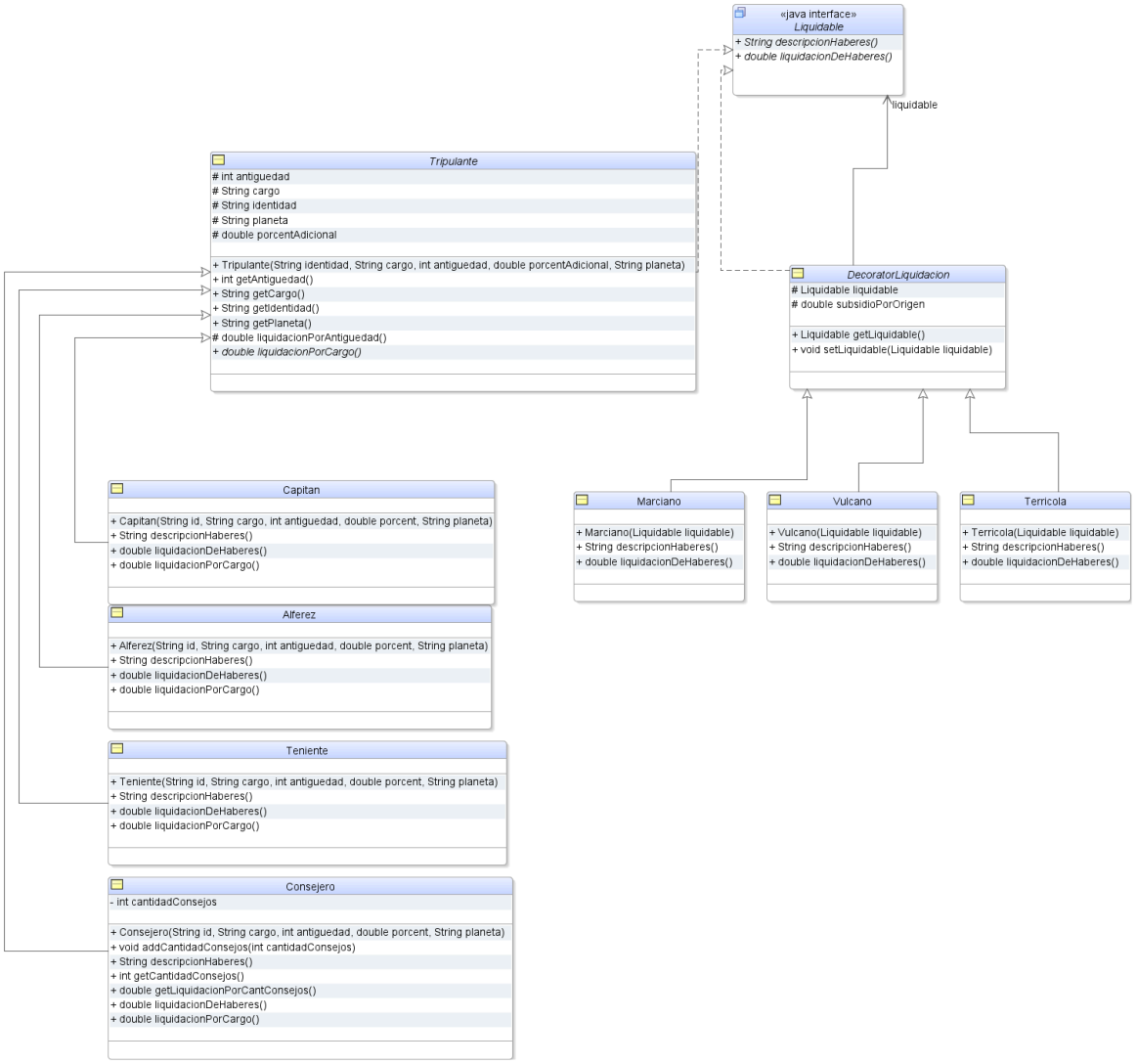
```
public class MisionFactory {

  /**
   * Método que devuelve una instancia de Mision según el tipo de misión.
   * <b>PRE:</b>
   * - tipoMision != null && !tipoMision.isEmpty()
   *
   * @param tipoMision El tipo de misión ("01", "02", "03").
   * @return Una instancia de Mision correspondiente al tipo especificado, o null si el tipo no es válido.
   */
  public static Mision getMision(String tipoMision) {
    switch (tipoMision) {
      case "01":
        return new Mision01();
      case "02":
        return new Mision02();
      case "03":
        return new Mision03();
      default:
        return null;
    }
  }
}
```

Este patrón nos permite encapsular la creación de los objetos de tipo nave y misión, delegando esta tarea a clases de tipo “fábrica”, permitiendo al usuario no conocer las clases concretas hijas de Nave y Misión.

Decorator

Para aplicar el patrón decorator se usó de referencia diapositivas con respecto al mismo, haciendo un paralelismo con los decoradores concretos (Marciano,Terrícola y Vulcano) quedando su diseño UML:



El patrón añade responsabilidades de forma dinámica con el objetivo de evitar la “explosion de clases”

Componentes Concretos: **Tripulante** (padre) y sus tres subclases (**Capitan, Alferez, Teniente y Consejero**) declaran valores bases en métodos implementados de la interfaz **Liquidable**, podría haberse llamado TripulanteInterface tranquilamente.

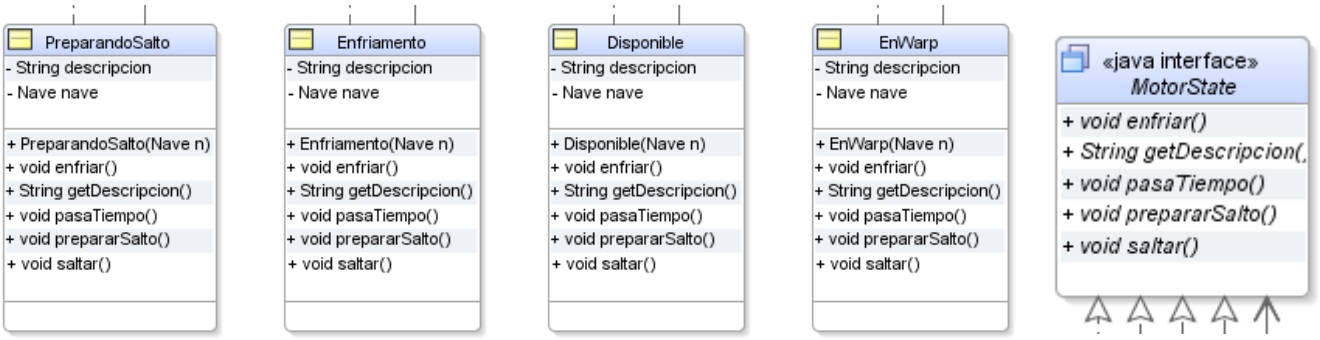
Decorador: **DecoratorLiquidacion** guarda una referencia de una subclase de **Tripulante**.

Decoradores Concretos: **Marciano, Vulcano,Terricola** extienden de **DecoratorLiquidacion** llaman a getLiquidable().metodointerface() y agregan funcionalidad. En sus constructores hacen super.setLiquidable(liquidable).

Hay que aclarar, nosotros decidimos que DecoratorLiquidacion no herede de Tripulante (para ello utilizamos la interfaz Liquidable, la cual implementan Tripulante y la clase decoradora) ya que queríamos mantener la lógica del Tripulante aislada del decorador, que para nada tenía que compartir atributos y métodos específicos de un Tripulante. Esta decisión, llevó a que la lista de tripulantes de la nave, en realidad, contenga Liquidables y no Tripulantes.

State

El patrón State permite encapsular el comportamiento de un objeto dependiendo del estado del mismo. En este trabajo lo usamos en el motor Warp de la nave, para poder pasar entre los distintos estados del motor (Disponible, Preparando salto, En Warp y Enfriamiento).



Estas clases “estado” heredan de la interfaz MotorState los métodos necesarios para las transiciones y es tarea de cada clase verificar si la transición se puede hacer o no. Por ejemplo, en la clase En Warp, solo deja hacer la transición a los estados Disponible y Enfriamiento; y al intentar hacer una transición invalida propaga la excepción TransicionErroneaException:

```
@Override
public void prepararSalto()  throws TransicionErroneaException {
    assert this.nave != null : "La nave no puede ser nula";
    throw new TransicionErroneaException("ERROR: no se puede preparar el salto en medio de un salto. Enfrie el motor o pase a disponible");
}

@Override
public void saltar()  throws TransicionErroneaException {
    assert this.nave != null : "La nave no puede ser nula";
    throw new TransicionErroneaException("ERROR: ya se esta en Warp. Enfrie el motor o pase a disponible");
}

@Override
public void enfriar() {
    this.nave.setEstado(new Enfriamento(this.nave));
    assert this.nave != null : "La nave no puede ser nula";
}

@Override
public void pasaTiempo() {
    this.nave.setEstado(new Disponible(this.nave));
    assert this.nave != null : "La nave no puede ser nula";
}
```